C++ Programming Problem Set

Module: Functions & Parameter Passing Mechanics

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Functions, Prototypes & Parameter Passing

Integrates: I/O, Conditionals, Loops, Pointers, References & Arrays

Instructions & Pedagogical Guidelines

This problem set evaluates your understanding of C++ function design. You will implement modular programs that test parameter passing strategies (Call by Value, Call by Address, Call by Reference), forward prototypes, stack frame safety, overload resolutions, and array processing. Write clean function declarations, definitions below `main()` where instructed, and adhere to modular DRY principles.

Level 1: Modular Design & The Three Parameter Passing Pillars

Problem 1: Temperature Multi-Converter & Scope Inspector

Call-by-Value | Math Operators | Functions

Write a function `double fahrenheitToCelsius(double f)` defined *below* `main()` (requiring a prototype declaration above `main()`). In `main()`, take a user temperature input in Fahrenheit inside a loop that runs until the user inputs `-999`. Demonstrate that modifying parameter `f` inside the conversion function leaves the variable in `main()` untouched due to Call-by-Value semantics.

Function Prototypes | Call by Value | Sentinel While Loop

Sample Input	Sample Output
`Input Temp: 98.6`	`Celsius: 37.00°C | Original Temp in main(): 98.60°F`

Problem 2: Parameter Passing Triangular Comparative Swap

Call-by-Address | Call-by-Reference | Pointers

Implement two distinct function versions that sort three numbers a, b, and c in ascending order in-place:
1. `void sortThreeAddress(int *p1, int *p2, int *p3)` using Call-by-Address (pointers & dereferencing).
2. `void sortThreeRef(int &r1, int &r2, int &r3)` using Call-by-Reference (aliases).
Call both functions in `main()` with different triplets and output the mutated values.

Call by Address | Call by Reference | Pointer Dereferencing

Sample Input	Sample Output
`Triplets: (45, 12, 29)`	`Address Version Sorted: 12, 29, 45` `Reference Version Sorted: 12, 29, 45`

Problem 3: Multiple Return Values via Reference & Address

Output Parameters | Control Structures | Functions

Functions in C++ can only return a single value via the `return` statement. Design a function `bool computeDivision(int dividend, int divisor, int &quotient, int *remainder)`. The function returns `false` if `divisor == 0` (guarding against divide-by-zero). If valid, it computes quotient (stored via reference) and remainder (stored via pointer), returning `true`.

Output Parameters | Error Flag Returns | Mixed Passing Types

Sample Inputs	Sample Outputs
`Dividend: 27, Divisor: 4`	`Status: SUCCESS | Quotient: 6 | Remainder: 3`

Level 2: Function Overloading, Arrays & Recursion Mechanics

Problem 4: Polymorphic Area Calculator (Function Overloading)

Function Overloading | Default Arguments | Math

Demonstrate C++ compile-time polymorphism by creating an overloaded function family named `calculateArea`:
- `double calculateArea(double radius)` for a circle.
- `double calculateArea(double length, double width)` for a rectangle.
- `double calculateArea(double base, double height, bool isTriangle)` for a triangle.

Integrate default arguments where applicable and call all three overloads based on a user menu prompt.

```
// Overloaded Prototypes (Same name, unique parameter signatures): double calculateArea(double radius); double calculateArea(double length, double width); double calculateArea(double base, double height, bool isTriangle);
```

Menu Selection	Sample Output
`Choice: 2 (Rectangle, len=5.0, wid=4.0)`	`Calculated Rectangle Area: 20.00`

Problem 5: In-Place Array Reverser & Search Engine

Array Parameters | Pass-by-Address | Modularity

Arrays decay into pointers when passed to functions. Write two modular functions:
1. `void reverseArray(int *arr, int size)` that reverses the array in-place using two-pointer swap logic.
2. `int linearSearch(const int arr[], int size, int target, int &comparisons)` that searches for an element, tracking comparison count via reference parameter.

Test both on an user-entered array of size N.

Array Decay to Pointers | Const Array Protection | Two-Pointer Swap

Sample Input	Sample Output
`Array: [10, 20, 30, 40, 50], Search target: 40`	`Reversed Array: [50, 40, 30, 20, 10]` `Target Found at Index 1 | Comparisons: 2`

Problem 6: Recursive Stack Trace & Factorial Visualizer

Recursion | Call Stack Frames | Base Cases

Write a recursive function `unsigned long long factorial(int n, int depth = 0)`. To visualize Call Stack Activation Records during execution, print indentation spaces proportional to `depth` when entering and returning from each stack frame. Handle negative inputs by returning 0 and define `n = 0` as the base case.

Call Stack Frames | Base Case vs Recurrence | Default Argument Depth

Sample Input	Sample Stack Visualization Output

`n = 3`	`-> Entering factorial(3)`
	`  -> Entering factorial(2)`
	`  -> Entering factorial(1)`
	`  Result: 6`

Level 3: Stack Safety, Static Storage & Comprehensive Capstone

Problem 7: The Dangling Reference & Static Lifetime Fix

Stack Frame Safety | Static Local Variables | Return-by-Ref

Construct a program demonstrating the classic "Dangling Reference" memory flaw. Write a flawed function `int& getStackCounterBad()` that returns a reference to a local variable (and explain why accessing it causes undefined behavior). Then write a corrected version `int& getStackCounterFixed()` using a `static` local variable. Demonstrate in `main()` how persistent static storage allows safe returning by reference and state retention across calls.

```
// Flawed vs Fixed Return-by-Reference: int& badFunc() { int temp = 0; return temp; } // WARNING: Dangling reference! int& goodFunc() { static int count = 0; return count; } // SAFE: Static storage duration.
```

Call Sequence	Observed Output
`Call goodFunc()++ three times in main()`	`Counter Persists: 1, 2, 3 (Safe static memory retention)`

Problem 8: Const Reference Performance vs Primitive Value Benchmark

Const References | String Overhead | Call Efficiency

Create a benchmark tool comparing pass-by-value vs pass-by-const-reference for strings. Implement:
1. `void processStringByValue(std::string text)`
2. `void processStringByConstRef(const std::string &text)`
In `main()`, prompt for a long paragraph. Call both functions inside a loop running 100,000 iterations. Explain why passing primitive types (`int`, `char`) by value is optimal while passing non-primitive strings by `const &` eliminates memory copy overhead while ensuring read-only immutability.

Const Reference Immutability | Copy Constructor Cost | Parameter Type Selection Rules

Parameter Strategy	Copy Behavior
`processStringByValue(s)`	`Deep copy constructed on stack frame for every invocation.`
`processStringByConstRef(s)`	`Zero-copy alias passed; original string guaranteed read-only.`

Problem 9: Cumulative Capstone: Modular Student Grade Management System

Comprehensive Integration | Functions | Arrays & Structs

Build a complete modular grading system using custom functions for each task:
1. `void inputGrades(int grades[], int size)` — Call-by-Address input loop.
2. `double computeStats(const int grades[], int size, int &highest, int &lowest)` — Array processing returning average directly, and min/max via reference.
3. `void curveGrades(int grades[], int size, int curveAmount = 5)` — Default argument adjustment.
4. `void printReport(const int grades[], int size, double avg, int hi, int lo)` — Const summary display.
Structure all logic cleanly with function declarations above `main()` and definitions below.

Modular Decomposition | Const Array Parameters | Default Arguments | Complete Module Capstone

Input Grades	Curve Applied	Report Output
`[72, 85, 90, 64, 78]`	`Default (+5)`	`Curved Avg: 82.8 | New Max: 95 | New Min: 69`